DEPLOYMENT AND MAINTENANCE MANUAL

Laravel Deployment on IIS Windows Server 2016

MariaDB · PHP 8.3 · FastCGI · URL Rewrite · Inertia and React

This manual tells you how to install, configure, and repair a Laravel application on Microsoft IIS. It includes the MariaDB commands, the PHP 8.3 settings, the IIS settings, and the troubleshooting records from the ATS deployment.

Item	Data
Document type	Combined deployment guide and implementation record
Language rules	ASD-STE100 Simplified Technical English
Platform	Windows Server 2016, IIS 10, PHP 8.3 or later, MariaDB
Application	Laravel with Inertia and React
Issue date	July 2026
Applicability	Production and test servers

WARNING — Do not point the IIS website to the Laravel project root. Point the website to the `public` folder only. The project root contains the `.env` file and other secret data.

About This Manual

Purpose

This manual gives one set of instructions for these tasks:

- Installation of IIS, PHP 8.3, Composer, and MariaDB on Windows Server 2016.
- Installation and configuration of a Laravel application that uses Inertia and React.
- Operation of the MariaDB command line for databases, users, privileges, and backups.
- Correction of the errors that occurred during the ATS deployment.

Language rules

This manual uses ASD-STE100 Simplified Technical English. These rules apply:

- Each instruction is one sentence. Procedural sentences have 20 words or less.
- Descriptive sentences have 25 words or less. Each paragraph has six sentences or less.
- The text uses the active voice and the present tense.
- One word has one meaning. The same word is used for the same thing in all sections.
- Warnings and cautions come before the related instructions.
- Technical names and technical verbs of the product keep their correct form. Examples are `web.config`, `FastCgiModule`, and `artisan`.

How to use this manual

If you must...	Read...
Build a new server	Sections 1 to 14, in sequence
Install the database only	Section 7
Find MariaDB commands quickly	Section 7.5 to Section 7.11
Release a new version of the code	Section 13 and Section 15
Repair an error	Section 16 and Section 17
Do the final checks	Section 19 and Section 20

Definitions and abbreviations

Term	Definition
ACL	Access control list. The list of Windows permissions on a folder or a file.
Application pool	The IIS process group that runs one website. It has its own Windows identity.
ATS	Assignment Tracking System. The example application in this manual.
FastCGI	The IIS module that runs the PHP program for each request.
Inertia	The library that connects the Laravel server code to the React pages.
MariaDB	The database server. It uses the MySQL protocol and the MySQL PDO driver.
NTS	Non-Thread-Safe. The PHP build type for IIS and FastCGI.
URL Rewrite	The IIS module that sends non-file requests to `index.php`.
Vite	The tool that builds the production JavaScript and CSS files.
Ziggy	The package that gives the Laravel `route()` helper to JavaScript.

Contents

1. Before You Start

WARNING — Windows Server 2016 and IIS 10 on Windows Server 2016 come to the end of extended support on 12 January 2027. The platform can host Laravel in July 2026. Make a plan to move the application to Windows Server 2022 or Windows Server 2025 before that date.

CAUTION — Do not install a public web application on a domain controller. Use a dedicated member server or an isolated virtual machine.

1.1 Data that you must record

Write down these items before you make changes to the server:

- The IIS bindings, the certificates, and the application pool identities.
- The PHP folder path and the loaded `php.ini` path.
- The MariaDB service name, the port, and the account names.
- The scheduled tasks that start Laravel commands.

1.2 Backups that you must make

1. Make a snapshot of the virtual machine, or export the IIS configuration.
2. Copy the application folder to a safe location.
3. Make a dump of the database. Section 7.10 gives the command.
4. Copy the production `.env` file to a safe location.
5. Test the restore procedure before you continue.

1.3 Names and paths in this manual

The examples use the values in the table. The ATS column shows the values from the deployment record.

Item	Example value	ATS value
Application folder	C:\Sites\MyLaravelApp	C:\Sites\ats
Public web root	C:\Sites\MyLaravelApp\public	C:\Sites\ats\public
PHP folder	C:\PHP	C:\PHP
Application pool	MyLaravelAppPool	ATS
Pool identity	IIS AppPool\MyLaravelAppPool	IIS AppPool\ATS
Website name	MyLaravelApp	ATS
Binding	HTTPS on port 443, app.example.com	HTTP on port 8080
Database name	my_laravel_app	ats
Database account	laravel_app@127.0.0.1	ats_admin@127.0.0.1
Database server	MariaDB on 127.0.0.1, port 3306	MariaDB on 127.0.0.1, port 3306
Frontend	Inertia and React, built with Vite	Inertia and React, built with Vite

2. Deployment Architecture

```
Users on the internet or on the LAN
        |
        v
IIS site binding (HTTP or HTTPS)
        |
        v
IIS URL Rewrite  -->  public\index.php
        |
        v
PHP 8.3 x64 NTS through FastCGI
        |
        v
Laravel application
        |
        +--> MariaDB (database, cache, sessions, queue)
        +--> storage folder (logs, files, cache)
        +--> mail server
```

Component	Function
IIS website	It receives the HTTP requests. Its physical path is the `public` folder.
URL Rewrite	It sends all requests for non-file paths to `index.php`.

FastCGI	It starts `php-cgi.exe` and gives the request to PHP.
Laravel	It finds the route, runs the controller, and makes the response.
MariaDB	It keeps the application data, the sessions, the cache, and the queue jobs.
Vite build	It makes the files in `public\build`. IIS sends these files directly.

WARNING — Do not use `php artisan serve` as the production web server. It is a development server. IIS and FastCGI must serve the production traffic.

3. Prerequisites and Compatibility

3.1 Required components

Component	Correct choice	Reason
Operating system	Windows Server 2016 with all patches	IIS 10 is included. The platform is near the end of support.
Laravel	Laravel 13	Laravel 13 needs PHP 8.3 or later.
PHP	PHP 8.3 or later, x64, Non-Thread-Safe	PHP gives the NTS build for IIS and FastCGI.
IIS modules	CGI (FastCGI) and URL Rewrite 2.0	FastCGI runs PHP. URL Rewrite sends routes to `index.php`.
Composer	Composer 2	It installs the production PHP packages.
Database	MariaDB 10.6 or later	The application uses the MySQL PDO driver.
Node.js	Node.js 20 or later	Node.js is necessary only when you build the Vite assets on the server.
C++ runtime	Visual C++ Redistributable 2015-2022 (x64)	The PHP Windows build needs this runtime.

3.2 PHP 8.3 extensions for Laravel

Laravel needs these PHP extensions: Ctype, cURL, DOM, Fileinfo, Filter, Hash, Mbstring, OpenSSL, PCRE, PDO, Session, Tokenizer, and XML.

The MariaDB connection also needs the `pdo_mysql` extension. Add the `mysqli` extension for the diagnostic tools.

```
C:\PHP\php.exe -v
C:\PHP\php.exe -m
C:\PHP\php.exe -m | findstr /I "pdo_mysql mbstring openssl curl fileinfo zip"
```

NOTE — Run `composer check-platform-reqs` in the project folder after `composer install`. This command finds a missing extension or a wrong PHP version before IIS gets traffic.

4. Installation of IIS and the Role Services

4.1 Installation with PowerShell

Open Windows PowerShell as Administrator. Then run this command:

```
Install-WindowsFeature Web-Server,Web-CGI,Web-Static-Content,Web-Default-Doc,Web-Http-Errors,
  Web-Http-Logging,Web-RequestMonitor,Web-Filtering,Web-Stat-Compression,Web-Mgmt-Console
  -IncludeManagementTools
```

Then check the result and start the web service:

```
Get-WindowsFeature Web-Server,Web-CGI,Web-Mgmt-Console
Start-Service W3SVC
Set-Service W3SVC -StartupType Automatic
```

4.2 Installation of URL Rewrite 2.0

1. Download URL Rewrite 2.0 from the Microsoft IIS website.
2. Install the module.
3. Close IIS Manager and open it again.
4. Make sure that the URL Rewrite icon is in the Features View.

CAUTION — IIS shows the error HTTP 500.19 when it cannot read the `<rewrite>` element. The usual cause is a missing URL Rewrite module. Install the module and open IIS Manager again.

4.3 Folders for PHP and for the logs

```
New-Item -ItemType Directory -Force C:\PHP
New-Item -ItemType Directory -Force C:\Logs
New-Item -ItemType File      -Force C:\Logs\php-errors.log
```

5. Installation and Setup of PHP 8.3

5.1 Selection of the PHP build

Download a PHP 8.3 (or later) Windows ZIP file from the official PHP website for Windows. Select these properties:

- x64, for a 64-bit Windows Server and a 64-bit application pool.
- Non-Thread-Safe (NTS), because PHP gives this build for IIS and FastCGI.
- A version that agrees with the `composer.json` and `composer.lock` files.

1. Install the Visual C++ Redistributable for Visual Studio 2015-2022 (x64).
2. Extract the ZIP file to `C:\PHP`.

> **CAUTION** — Do not put files from two different PHP versions in the same folder. Delete the old folder first, or use a new folder.

5.2 The php.ini file

Make the production configuration file:

```
Copy-Item C:\PHP\php.ini-production C:\PHP\php.ini
notepad C:\PHP\php.ini
```

Set these values. The values are a safe production baseline. Change the upload sizes and the timeouts for your application.

```
; ---- Core paths and locale ----
extension_dir = "C:\PHP\ext"
date.timezone = Africa/Kampala

; ---- IIS and FastCGI ----
cgi.force_redirect = 0
cgi.fix_pathinfo = 1
fastcgi.impersonate = 1
fastcgi.logging = 0

; ---- Security and errors ----
expose_php = Off
display_errors = Off
display_startup_errors = Off
log_errors = On
error_log = "C:\Logs\php-errors.log"

; ---- Limits ----
memory_limit = 256M
max_execution_time = 120
post_max_size = 32M
upload_max_filesize = 32M
max_file_uploads = 20
max_input_vars = 3000
session.save_path = "C:\Windows\Temp"

; ---- Extensions for Laravel and MariaDB ----
extension=php_curl.dll
extension=php_fileinfo.dll
extension=php_mbstring.dll
extension=php_openssl.dll
extension=php_pdo_mysql.dll
extension=php_mysqli.dll
extension=php_intl.dll
extension=php_zip.dll
extension=php_gd.dll
extension=php_exif.dll
extension=php_sodium.dll

; ---- OPcache ----
zend_extension=php_opcache.dll
opcache.enable=1
opcache.enable_cli=0
opcache.memory_consumption=128
opcache.interned_strings_buffer=16
opcache.max_accelerated_files=20000
opcache.validate_timestamps=1
opcache.revalidate_freq=2

; ---- MySQL/MariaDB client defaults ----
mysqli.default_port = 3306
pdo_mysql.default_socket =
```

> **NOTE** — Microsoft gives `cgi.force_redirect=0`, `cgi.fix_pathinfo=1`, `fastcgi.impersonate=1`, and `fastcgi.logging=0` as the correct IIS values. Microsoft also recommends a writable PHP error log file.

5.3 The PHP folder in the system PATH

```
[Environment]::SetEnvironmentVariable(
```

```
  "Path",
  [Environment]::GetEnvironmentVariable("Path","Machine") + ";C:\PHP",
  "Machine"
)
```

Close PowerShell. Open PowerShell again. Then do these checks:

```
php --ini
php -v
php -m
```

The output of `php --ini` must show `C:\PHP\php.ini` as the loaded file.

5.4 The FastCGI handler in IIS

Do these steps in IIS Manager at the server level:

1. Open **Handler Mappings**.
2. Select **Add Module Mapping**.
3. Set **Request path** to `*.php`.
4. Set **Module** to `FastCgiModule`.
5. Set **Executable** to `C:\PHP\php-cgi.exe`.
6. Set **Name** to `PHP-FastCGI`.
7. Select **Yes** when IIS asks to make the FastCGI application.

> **WARNING** — Set the executable to `php-cgi.exe`. Do not set the executable to `php.exe`. IIS gives a FastCGI error with `php.exe`.

Then open **FastCGI Settings** and edit the entry for `C:\PHP\php-cgi.exe`:

Setting	Start value
Instance MaxRequests	10000
Activity Timeout	120 seconds
Request Timeout	300 seconds for long imports. Use a lower value for other applications.
Idle Timeout	300 seconds
Environment variable	PHP_FCGI_MAX_REQUESTS=10000

Recycle the application pool after each change to PHP or to FastCGI.

5.5 Test of the PHP installation

1. Make the file `C:\Sites\ats\public\test.php` with this content: `<?php phpinfo();`
2. Open `http://localhost:8080/test.php` on the server.
3. Make sure that the PHP version is 8.3 or later.
4. Make sure that the loaded configuration file is `C:\PHP\php.ini`.
5. Find `pdo_mysql` in the list of extensions.
6. Delete `test.php` immediately.

> **WARNING** — Do not keep `phpinfo` or `test.php` on a production server. The page shows the server configuration to all users.

6. Composer and the Frontend Assets

6.1 Installation of Composer 2

1. Download the official Composer Windows installer.
2. Select `C:\PHP\php.exe` when the installer asks for the PHP program.
3. Let the installer add Composer to the PATH.
4. Open a new PowerShell window.

```
composer --version
composer diagnose
```

6.2 Location for the build of the frontend assets

Method	Use	Commands
Build before the deployment	Preferred for production. Node.js stays off the server.	npm ci npm run build Copy public\build to the server
Build on the server	Permitted for an internal server or a manual deployment.	npm ci npm run build

NOTE — An Inertia and React application needs the file `public\build\manifest.json`. Laravel gives a Vite manifest error when this file is absent.

7. MariaDB Installation and Commands

The environment uses MariaDB. Laravel connects to MariaDB with the MySQL PDO driver. This section gives the installation steps and all the necessary commands for daily operation.

7.1 Installation of the MariaDB server

1. Download the MariaDB MSI installer (x64) for Windows.
2. Start the installer as Administrator.
3. Set a strong password for the `root` account.
4. Clear the option that permits remote access for `root`.
5. Select **Use UTF8 as default server character set**.
6. Set the service name to `MariaDB` and set the port to `3306`.
7. Complete the installation.

Then check the service:

```
Get-Service MariaDB
Start-Service MariaDB
Set-Service MariaDB -StartupType Automatic
```

7.2 The MariaDB folder in the system PATH

The client programs are in the `bin` folder of MariaDB. Add this folder to the PATH:

```
[Environment]::SetEnvironmentVariable(
  "Path",
  [Environment]::GetEnvironmentVariable("Path","Machine") + ";C:\Program Files\MariaDB 11.4\bin",
  "Machine"
)
```

Change the version number in the path to the installed version. Then open a new window and do this check:

```
mariadb --version
mariadb -u root -p -e "SELECT VERSION();"
```

7.3 The server configuration file

The configuration file is `C:\Program Files\MariaDB 11.4\data\my.ini`. Set these values in the `[mysqld]` section:

```
[mysqld]
port                        = 3306
bind-address                = 127.0.0.1
character-set-server        = utf8mb4
collation-server            = utf8mb4_unicode_ci
default_storage_engine      = InnoDB
innodb_buffer_pool_size     = 1G
max_allowed_packet          = 64M
max_connections             = 200
innodb_file_per_table       = 1
sql_mode                    = STRICT_TRANS_TABLES,NO_ENGINE_SUBSTITUTION

[client]
default-character-set       = utf8mb4
```

Restart the service after each change:

```
Restart-Service MariaDB
```

> **CAUTION** — Set `bind-address` to `127.0.0.1` when the database and the application are on the same server. This value stops all network connections to the database from other computers.

7.4 First security steps

Run the security tool one time after the installation:

```
mariadb-secure-installation
```

Give these answers:

- Set a root password: **yes**, if the password is not set.
- Remove anonymous users: **yes**.
- Disallow root login remotely: **yes**.
- Remove the test database: **yes**.
- Reload the privilege tables: **yes**.

7.5 Connection to the MariaDB command line

Task	Command (Windows command prompt or PowerShell)
Connect as root	`mariadb -u root -p`
Connect as the application user	`mariadb -u ats_admin -p -h 127.0.0.1 -P 3306`
Connect and select a database	`mariadb -u ats_admin -p ats`
Run one command and exit	`mariadb -u root -p -e "SHOW DATABASES;"`
Use the full path	`"C:\Program Files\MariaDB 11.4\bin\mariadb.exe" -u root -p`
Exit the command line	`EXIT;` or `QUIT;`

NOTE — The programs `mysql.exe` and `mysqldump.exe` are also in the MariaDB `bin` folder. They are alternative names for `mariadb.exe` and `mariadb-dump.exe`. The commands are the same.

WARNING — Do not put the password in the command line with the `-p` option (for example, `-pSecret123`). Windows keeps the command in the console history. Type the password at the prompt.

7.6 Commands to show data

Use these commands at the `MariaDB [(none)]>` prompt. Each command ends with a semicolon.

```
-- Show the server version and the connection status
SELECT VERSION();
STATUS;

-- Show all databases
SHOW DATABASES;

-- Select a database for the session
USE ats;

-- Show the current database
SELECT DATABASE();

-- Show all tables in the current database
SHOW TABLES;

-- Show the tables of a different database
SHOW TABLES FROM ats;

-- Show the columns of a table
DESCRIBE users;
SHOW COLUMNS FROM users;

-- Show the command that makes a table
SHOW CREATE TABLE users\G

-- Show the indexes of a table
SHOW INDEX FROM users;

-- Show all user accounts
SELECT User, Host FROM mysql.user ORDER BY User;

-- Show the size of each table in megabytes
SELECT table_name,
       ROUND((data_length + index_length) / 1024 / 1024, 2) AS size_mb
FROM   information_schema.tables
WHERE  table_schema = 'ats'
ORDER  BY size_mb DESC;

-- Show the character set and the collation of the database
SELECT default_character_set_name, default_collation_name
FROM   information_schema.schemata
WHERE  schema_name = 'ats';

-- Show the active connections
SHOW PROCESSLIST;
```

7.7 Commands to make and to delete a database

```
-- Make the database with the correct character set
CREATE DATABASE IF NOT EXISTS ats
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

-- Change the character set of an existing database
ALTER DATABASE ats
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;
```

```
-- Delete the database and all its tables
DROP DATABASE IF EXISTS ats;

-- Delete one table
DROP TABLE IF EXISTS old_sessions;

-- Delete all rows of a table but keep the structure
TRUNCATE TABLE cache;
```

WARNING — The command `DROP DATABASE` deletes all the data immediately. MariaDB does not ask for a confirmation. Make a dump of the database before you use this command.

7.8 Commands to make and to delete a user

CAUTION — Do not use the MySQL 8 syntax `IDENTIFIED WITH caching_sha2_password BY '...'`. MariaDB does not use this authentication plugin in the default installation. It gives a syntax error. Use `IDENTIFIED BY`.

```
-- Make the application account (correct syntax for MariaDB)
CREATE USER 'ats_admin'@'127.0.0.1'
  IDENTIFIED BY 'YOUR_STRONG_PASSWORD';

-- Make an account for a read-only report tool
CREATE USER 'ats_reader'@'127.0.0.1'
  IDENTIFIED BY 'ANOTHER_STRONG_PASSWORD';

-- Change the password of an account
ALTER USER 'ats_admin'@'127.0.0.1'
  IDENTIFIED BY 'NEW_STRONG_PASSWORD';

-- Alternative command for the password
SET PASSWORD FOR 'ats_admin'@'127.0.0.1' = PASSWORD('NEW_STRONG_PASSWORD');

-- Change the name of an account
RENAME USER 'ats_admin'@'127.0.0.1' TO 'ats_app'@'127.0.0.1';

-- Delete an account
DROP USER IF EXISTS 'ats_reader'@'127.0.0.1';

-- Apply the changes to the privilege tables
FLUSH PRIVILEGES;
```

NOTE — The host part of the account name is important. The account `'ats_admin'@'127.0.0.1'` is not the same as `'ats_admin'@'localhost'`. Laravel with `DB_HOST=127.0.0.1` uses the TCP host `127.0.0.1`. Make the account for that exact host.

7.9 Commands to add and to remove privileges

```
-- Give all privileges on one database only
GRANT ALL PRIVILEGES ON ats.* TO 'ats_admin'@'127.0.0.1';

-- Give the minimum privileges for a Laravel application with migrations
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, ALTER, INDEX, REFERENCES
  ON ats.* TO 'ats_admin'@'127.0.0.1';

-- Give read-only access
GRANT SELECT ON ats.* TO 'ats_reader'@'127.0.0.1';

-- Give a privilege on one table only
GRANT SELECT, UPDATE ON ats.users TO 'ats_reader'@'127.0.0.1';

-- Apply the changes
FLUSH PRIVILEGES;

-- Show the privileges of an account
SHOW GRANTS FOR 'ats_admin'@'127.0.0.1';

-- Show the privileges of the current account
SHOW GRANTS FOR CURRENT_USER();

-- Remove one privilege
REVOKE DELETE ON ats.* FROM 'ats_reader'@'127.0.0.1';

-- Remove all privileges
REVOKE ALL PRIVILEGES, GRANT OPTION FROM 'ats_reader'@'127.0.0.1';

FLUSH PRIVILEGES;
```

WARNING — Do not give the privilege `ALL PRIVILEGES ON *.*` to an application account. Give privileges on the application database only. This limit protects the other databases on the server.

7.10 Commands to export a database (backup)

Run these commands in the Windows command prompt. Do not run them at the MariaDB prompt.

```
:: Export one database with the structure and the data
mariadb-dump -u ats_admin -p --single-transaction --routines --events ^
  --default-character-set=utf8mb4 ats > C:\Backups\ats_2026-07-27.sql

:: Export the structure only (no rows)
mariadb-dump -u ats_admin -p --no-data ats > C:\Backups\ats_schema.sql

:: Export the data only (no CREATE TABLE commands)
mariadb-dump -u ats_admin -p --no-create-info ats > C:\Backups\ats_data.sql

:: Export one table
mariadb-dump -u ats_admin -p ats users > C:\Backups\ats_users.sql

:: Export more than one table
mariadb-dump -u ats_admin -p ats users assignments > C:\Backups\ats_two_tables.sql

:: Export all the databases on the server
mariadb-dump -u root -p --all-databases > C:\Backups\all_databases.sql

:: Export and compress with 7-Zip
mariadb-dump -u ats_admin -p ats | 7z a -si C:\Backups\ats.sql.gz
```

The equivalent PowerShell command uses a redirection with the correct encoding:

```
cmd /c "mariadb-dump -u ats_admin -p ats > C:\Backups\ats_2026-07-27.sql"
```

> **CAUTION** — Do not use the PowerShell operator `>` directly with `mariadb-dump`. PowerShell writes the file in UTF-16. The import operation then fails. Use `cmd /c`, as shown above.

7.11 Commands to import a database (restore)

```
:: Make the empty database first
mariadb -u root -p -e "CREATE DATABASE IF NOT EXISTS ats CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;"

:: Import the dump file
mariadb -u ats_admin -p ats < C:\Backups\ats_2026-07-27.sql

:: Import all the databases from a full dump
mariadb -u root -p < C:\Backups\all_databases.sql
```

You can also import a file at the MariaDB prompt. Use forward slashes in the path:

```
USE ats;
SOURCE C:/Backups/ats_2026-07-27.sql;
```

After the import, do these checks:

```
mariadb -u ats_admin -p -e "USE ats; SHOW TABLES;"
mariadb -u ats_admin -p -e "SELECT COUNT(*) FROM ats.users;"
```

7.12 Configuration for the ATS deployment

These commands made the ATS database and the ATS account:

```
CREATE USER 'ats_admin'@'127.0.0.1'
  IDENTIFIED BY 'YOUR_STRONG_PASSWORD';

CREATE DATABASE IF NOT EXISTS ats
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

GRANT ALL PRIVILEGES ON ats.*
  TO 'ats_admin'@'127.0.0.1';

FLUSH PRIVILEGES;

SHOW GRANTS FOR 'ats_admin'@'127.0.0.1';
```

7.13 Test of the connection from PHP

Test the database connection with PHP before you start Laravel:

```
:: Run this line in the Windows command prompt (cmd)
php -r "$pdo = new PDO('mysql:host=127.0.0.1;port=3306;dbname=ats','ats_admin','YOUR_STRONG_PASSWORD'); echo 'Connection OK';"
```

PowerShell replaces a text that starts with the dollar sign. Use `cmd /c` in PowerShell:

```
cmd /c "php -r \"$pdo = new PDO('mysql:host=127.0.0.1;dbname=ats','ats_admin','YOUR_STRONG_PASSWORD'); echo 'Connection OK';\""
```

Then test the connection through Laravel:

```
Set-Location C:\Sites\ats
php artisan db:show
php artisan migrate:status
```

7.14 Common MariaDB errors

Message	Cause	Correction
SQLSTATE[HY000] [1045] Access denied	The password or the account name is wrong.	Check `DB_USERNAME` and `DB_PASSWORD`. Set the password again with `ALTER USER`.
SQLSTATE[HY000] [1044] Access denied for user to database	The account has no privileges on the database.	Run the `GRANT` command for the exact user and host combination. Then run `FLUSH PRIVILEGES`.
SQLSTATE[HY000] [1049] Unknown database	The database does not exist.	Make the database with `CREATE DATABASE`.
SQLSTATE[HY000] [2002] Connection refused	The service is stopped, or the port is wrong.	Run `Start-Service MariaDB`. Check the port in `my.ini`.
could not find driver	The PHP extension `pdo_mysql` is off.	Remove the semicolon before `extension=php_pdo_mysql.dll`. Recycle the application pool.
You have an error in your SQL syntax near 'caching_sha2_password'	The command uses the MySQL 8 syntax.	Use `CREATE USER ... IDENTIFIED BY '...'`.
Specified key was too long	The index is longer than the maximum key length.	Add `Schema::defaultStringLength(191);` to `AppServiceProvider::boot()`.

8. Installation of the Laravel Application

8.1 The application folder

```
New-Item -ItemType Directory -Force C:\Sites\ats
```

Copy the project with Git, with a deployment ZIP file, or with a CI/CD agent. Do not copy the local `vendor` folder, the `node_modules` folder, the development `.env` file, or other secret files.

8.2 The production PHP packages

```
Set-Location C:\Sites\ats
composer install --no-dev --prefer-dist --optimize-autoloader --no-interaction
composer check-platform-reqs
```

CAUTION — Deploy the `composer.lock` file from source control. Do not run `composer update` on the production server. That command can install unexpected package versions.

8.3 The production .env file

```
Copy-Item .env.example .env
notepad .env
```

Set these values for a MariaDB installation on the same server:

```
APP_NAME="Assignment Tracking System"
APP_ENV=production
APP_KEY=
APP_DEBUG=false
APP_URL=http://ats.example.go.ug:8080

LOG_CHANNEL=stack
LOG_LEVEL=warning

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=ats
DB_USERNAME=ats_admin
DB_PASSWORD="YOUR_STRONG_PASSWORD"
DB_CHARSET=utf8mb4
DB_COLLATION=utf8mb4_unicode_ci

CACHE_STORE=database
SESSION_DRIVER=database
SESSION_SECURE_COOKIE=false
QUEUE_CONNECTION=database
```

Set `SESSION_SECURE_COOKIE=true` when the site uses HTTPS.

Then make the application key for a new installation:

```
php artisan key:generate --force
```

WARNING — Do not change `APP_KEY` on an existing system. A new key makes the encrypted cookies invalid. It can also make the encrypted data in the database unreadable. Keep the production key during each new release.

WARNING — Do not put the `.env` file in the `public` folder. Do not commit the `.env` file to source control.

8.4 Checks from the command line

```
php artisan about
php artisan config:show app
php artisan db:show
php artisan migrate:status
```

9. NTFS Permissions

Laravel writes to the `storage` folder and to the `bootstrap\cache` folder. The application pool identity must have Modify access on these two folders only.

9.1 Permission commands

Make the application pool first (Section 10.1). Then run these commands:

```
$AppPath      = "C:\Sites\ats"
$PoolIdentity = "IIS AppPool\ATS"

icacls $AppPath /inheritance:r
icacls $AppPath /grant "Administrators:(OI)(CI)(F)" /T
icacls $AppPath /grant "SYSTEM:(OI)(CI)(F)" /T
icacls $AppPath /grant "${PoolIdentity}:(OI)(CI)(RX)" /T
icacls "$AppPath\storage" /grant "${PoolIdentity}:(OI)(CI)(M)" /T
icacls "$AppPath\bootstrap\cache" /grant "${PoolIdentity}:(OI)(CI)(M)" /T
icacls "C:\Sites" /grant "${PoolIdentity}:(RX)"
iisreset
```

Give Modify access to an upload folder only when the application writes to that folder.

9.2 Permission rules

Location	Access for the pool identity
C:\Sites\ats (all files and folders)	Read and Execute
C:\Sites\ats\storage	Modify
C:\Sites\ats\bootstrap\cache	Modify
Upload folders (if used)	Modify
C:\Sites	Read and Execute (no inheritance)

WARNING — Do not give Full Control to `Everyone`, `Users`, `IUSR`, or the group `IIS_IUSRS`. These permissions remove the isolation between the websites.

NOTE — Open the **AUTHENTICATION** feature of the website. Edit **ANONYMOUS AUTHENTICATION** and select **APPLICATION POOL IDENTITY**. This setting makes the ACL and the PHP impersonation agree.

10. Application Pool and Website

10.1 The application pool

1. Open IIS Manager.
2. Select **Application Pools**, then select **Add Application Pool**.
3. Set **Name** to `ATS`.
4. Set **.NET CLR version** to `No Managed Code`.
5. Set **Managed pipeline mode** to `Integrated`.
6. Open **Advanced Settings**.
7. Set **Enable 32-Bit Applications** to `False` for a 64-bit PHP build.
8. Set **Identity** to `ApplicationPoolIdentity`.

Use one application pool for each website. The virtual account gives isolation between the applications.

10.2 The website

1. Select **Sites**, then select **Add Website**.
2. Set **Site name** to `ATS`.
3. Set **Application pool** to `ATS`.
4. Set **Physical path** to `C:\Sites\ats\public`.
5. Set the binding. The ATS deployment uses HTTP on port 8080.
6. Start the website.

WARNING — The physical path must end with `public`. A path to the project root makes the `.env` file and the source code available to all users.

10.3 Other website settings

Feature	Value
Default document	index.php (delete the other entries)
Directory browsing	Off
Anonymous authentication	On, with the application pool identity
Handler mapping	*.php to FastCgiModule and php-cgi.exe
Error pages	DetailedLocalOnly

11. Rewrite Rules in public\web.config

Laravel routes such as `/login`, `/home`, and `/users/12` are not physical files. IIS must send these requests to `public\index.php`. Make the file `C:\Sites\ats\public\web.config` with this content:

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <system.webServer>

    <defaultDocument enabled="true">
      <files>
        <clear />
        <add value="index.php" />
      </files>
    </defaultDocument>

    <directoryBrowse enabled="false" />

    <rewrite>
      <rules>
        <rule name="Laravel Front Controller" stopProcessing="true">
          <match url="^(.*)$" ignoreCase="false" />
          <conditions logicalGrouping="MatchAll">
            <add input="{REQUEST_FILENAME}" matchType="IsFile"      negate="true" />
            <add input="{REQUEST_FILENAME}" matchType="IsDirectory" negate="true" />
          </conditions>
          <action type="Rewrite" url="index.php" appendQueryString="true" />
        </rule>
      </rules>
    </rewrite>

    <httpErrors errorMode="DetailedLocalOnly" existingResponse="PassThrough" />

    <httpProtocol>
      <customHeaders>
        <remove name="X-Powered-By" />
        <add name="X-Content-Type-Options" value="nosniff" />
        <add name="X-Frame-Options"        value="SAMEORIGIN" />
        <add name="Referrer-Policy"        value="strict-origin-when-cross-origin" />
      </customHeaders>
    </httpProtocol>

  </system.webServer>
</configuration>
```

The rule does not change the requests for real files and real folders. IIS sends the CSS files, the JavaScript files, and the images directly.

> **CAUTION** — Save `web.config` as UTF-8 without a byte order mark. A different encoding gives the error HTTP 500.19 with the code 0x8007000d.

> **WARNING** — Do not move `index.php` out of the `public` folder. Do not change the paths in `index.php`. Point IIS to the `public` folder instead.

11.1 Optional rule for HTTPS

Add this rule before the "Laravel Front Controller" rule. Add the rule only after the HTTPS binding operates correctly.

```
<rule name="Redirect to HTTPS" stopProcessing="true">
  <match url="(.*)" />
  <conditions>
    <add input="{HTTPS}" pattern="off" ignoreCase="true" />
  </conditions>
  <action type="Redirect"
          url="https://{HTTP_HOST}/{R:1}"
          redirectType="Permanent"
          appendQueryString="true" />
</rule>
```

> **CAUTION** — Test this rule carefully when a load balancer or a proxy stops the TLS connection. Set the trusted proxies in Laravel. Incorrect settings make an endless redirect loop.

11.2 Optional limit for large uploads

PHP and IIS have separate limits for the request size. Set the PHP values in `php.ini`. Then add this section to `web.config`. The value is in bytes. The example permits 32 MB.

```
<security>
  <requestFiltering>
    <requestLimits maxAllowedContentLength="33554432" />
  </requestFiltering>
</security>
```

12. HTTPS and Production Settings

12.1 The HTTPS binding

1. Import a valid TLS certificate into the Local Computer certificate store.
2. Open the **Bindings** dialog of the website.
3. Add a binding of the type `https` on port 443.
4. Select the host name and the certificate.
5. Select **Server Name Indication** when more than one HTTPS site uses the same IP address.
6. Open TCP port 443 in Windows Firewall and in the perimeter firewall.
7. Make sure that DNS gives the correct address for the host name.

12.2 Laravel production values

```
APP_ENV=production
APP_DEBUG=false
APP_URL=https://ats.example.go.ug
SESSION_SECURE_COOKIE=true
```

WARNING — Set `APP_DEBUG` to `false` on a production server. The debug page shows the environment values, the database credentials, and the file paths.

13. Migrations and Optimization

Run these commands for the first deployment and for each new release. Use an elevated terminal or a deployment account with write access to the Laravel cache folders.

```
Set-Location C:\Sites\ats

php artisan down --retry=60
composer install --no-dev --prefer-dist --optimize-autoloader --no-interaction
php artisan optimize:clear
php artisan migrate --force
php artisan storage:link
php artisan optimize
php artisan queue:restart
php artisan up
```

The command `php artisan optimize` makes the cache files for the configuration, the events, the routes, and the views.

> **CAUTION** — The command `php artisan storage:link` makes a symbolic link. Windows needs an elevated shell or the symbolic link privilege for this operation. Run the command one time. Then make sure that the folder `public\storage` exists.

13.1 Checks after the deployment

Open these addresses in a browser:

- `http://ats.example.go.ug:8080/` — the home page or the login page.
- `http://ats.example.go.ug:8080/up` — the Laravel health route. It gives HTTP 200.
- A route with a parameter, such as `/users/1`. This test shows that URL Rewrite operates.
- A file under `/build/assets/`. This test shows that IIS sends the static files.

Then run these commands:

```
php artisan about
php artisan route:list
php artisan migrate:status
php artisan db:show
Get-Content .\storage\logs\laravel.log -Tail 100
```

14. Scheduler and Queue Workers

14.1 The Laravel scheduler

Make a local service account with these permissions: Read and Execute on the project, and Modify on `storage` and `bootstrap\cache`. Then make a task in Windows Task Scheduler:

1. Select **Create Task**. Do not select **Create Basic Task**.
2. Select **Run whether user is logged on or not**.
3. Set the trigger to **Daily**. Set the repetition to 1 minute, indefinitely.
4. Set **Program** to `C:\PHP\php.exe`.
5. Set **Arguments** to `artisan schedule:run`.
6. Set **Start in** to `C:\Sites\ats`.
7. Select **Run task as soon as possible after a scheduled start is missed**.

Test the task manually:

```
Set-Location C:\Sites\ats
C:\PHP\php.exe artisan schedule:run -v
```

14.2 The queue worker

A queue worker must run continuously. Use a Windows service wrapper, or a scheduled task that starts at boot and restarts after a failure.

```
C:\PHP\php.exe C:\Sites\ats\artisan queue:work --sleep=3 --tries=3 --timeout=90 --max-time=3600
```

Restart the workers after each deployment. The workers then load the new code:

```
php artisan queue:restart
```

> **CAUTION** — Do not run the scheduler or the queue worker as `Administrator` or as `LocalSystem`. Use a dedicated account with the minimum privileges.

> **NOTE** — Do not keep a queue worker in an interactive PowerShell window. The worker stops when the session ends.

15. Repeatable Deployment Script

Save this script as `C:\Deploy\Deploy-Ats.ps1`. Change the paths for your server. Add your Git step or your file-copy step at the marked position.

```
param(
  [string]$AppPath  = "C:\Sites\ats",
  [string]$Php      = "C:\PHP\php.exe",
  [string]$Composer = "C:\ProgramData\ComposerSetup\bin\composer.bat"
)

$ErrorActionPreference = "Stop"
Set-Location $AppPath

function Run-Step([string]$Name, [scriptblock]$Action) {
  Write-Host "==> $Name" -ForegroundColor Cyan
  & $Action
  if ($LASTEXITCODE -ne 0) {
    throw "$Name failed with exit code $LASTEXITCODE"
  }
}

$DownSucceeded = $false

try {
  Run-Step "Enable maintenance mode" { & $Php artisan down --retry=60 }
  $DownSucceeded = $true

  # Insert the Git pull or the file-copy operation here.

  Run-Step "Install Composer dependencies" {
    & $Composer install --no-dev --prefer-dist --optimize-autoloader --no-interaction
  }
  Run-Step "Clear old caches"        { & $Php artisan optimize:clear }
  Run-Step "Run database migrations" { & $Php artisan migrate --force }
  Run-Step "Build Laravel caches"    { & $Php artisan optimize }
  Run-Step "Restart queue workers"   { & $Php artisan queue:restart }

  Write-Host "Deployment completed successfully." -ForegroundColor Green
}
catch {
  Write-Error $_
  throw
}
finally {
  if ($DownSucceeded) {
    & $Php artisan up
  }
}
```

WARNING — Make a database dump before a release with destructive migrations. Old code does not reverse a migration automatically. Keep the previous application files for a rollback.

16. Troubleshooting

16.1 Errors recorded during the ATS deployment

These errors occurred in sequence during the installation of the ATS application. The table gives the meaning of each error and the correction that was made.

Error	Meaning	Correction
MariaDB syntax error at `caching_sha2_password`	The `CREATE USER` command used the MySQL 8 syntax. MariaDB does not use that plugin.	The account was made again with `IDENTIFIED BY` . See Section 7.8.
SQLSTATE 1044 — access denied to the database	Laravel connected to MariaDB, but the account had no privileges on the `ats` database.	Database privileges were given to the exact user and host combination. See Section 7.9.
HTTP 401.3	IIS found the application folder, but the pool identity had insufficient NTFS access.	Read and Execute access was given to `IIS AppPool\ATS` . See Section 9.1.
HTTP 403.14	IIS read the `public` folder, but it did not know which file to open.	The default document was set to `index.php` . Directory browsing stays off.
HTTP 500.19 with the code 0x8007000d	IIS could not read `web.config` . The `<rewrite>` section was present, but the URL Rewrite module was not installed. A wrong file encoding gives the same result.	URL Rewrite 2.0 was installed. The file was saved as UTF-8 without a byte order mark.
HTTP 404 for `/home`	`index.php` ran and Laravel made a redirect to `/home` . IIS then looked for a folder with that name, because URL rewriting was not active.	The rewrite rule in Section 11 was added. IIS Manager was opened again.
JavaScript error: `route is not defined`	The login page calls `route(...)` , but the Ziggy route helper is not in the compiled module or in the global scope of the page.	See Section 17.

16.2 General troubleshooting table

Symptom	Probable cause	Correction
HTTP 500.19, the `<rewrite>` element is unknown	URL Rewrite is not installed.	Install URL Rewrite 2.0. Open IIS Manager again. Recycle the pool.
HTTP 500.19, access denied	IIS cannot read `web.config` or the site files.	Check the physical path. Give Read access to the pool identity.
HTTP 500.0, unknown FastCGI error	The PHP build is wrong, the VC++ runtime is absent, or `php.ini` has an error.	Run `php -v` . Read `C:\Logs\php-errors.log` . Check the x64 and NTS properties.
HTTP 401.3	The pool identity has insufficient NTFS access.	Run the `icacls` commands in Section 9.1. Set anonymous authentication to the pool identity.
HTTP 403.14 (directory listing refused)	The default document is absent.	Add `index.php` as the default document. Keep directory browsing off.
The home page operates, but each route gives HTTP 404	The rewrite rule is absent, or URL Rewrite is not active.	Put `web.config` in the `public` folder. Check the URL Rewrite feature.
Laravel error "Permission denied"	The pool identity has no Modify access on `storage` or `bootstrap\cache` .	Give Modify access to those two folders only.
Vite manifest not found	The frontend assets were not built, or `public\build` was not copied.	Run `npm ci` and `npm run build` , or copy the build folder from CI.
could not find driver	The PDO extension is off.	Turn on `pdo_mysql` in `php.ini` . Recycle the pool.
No application encryption key	`APP_KEY` is empty.	Run `php artisan key:generate --force` for a new system. Keep the existing key for a release.
Uploads fail at a fixed size	A PHP limit or an IIS limit is too low.	Set `upload_max_filesize` , `post_max_size` , and `maxAllowedContentLength` to the same value.
CSS and JavaScript load through HTTP on an HTTPS site	`APP_URL` , the proxy trust, or the cache is incorrect.	Set `APP_URL` to the HTTPS address. Set the trusted proxies. Run `php artisan optimize:clear` and then `php artisan optimize` .
Old code operates after a release	The Laravel cache or OPcache keeps the old files.	Run `php artisan optimize:clear` . Then run `iisreset` .

17. The Error "route is not defined"

This error is a frontend JavaScript error. It is not an IIS error, a PHP error, a MariaDB error, a password error, or a Laravel authentication error. The browser opens the login component, but the component calls `route('...')` without the Ziggy route helper.

17.1 Probable cause

The production JavaScript bundle was built while the login component used `route()` as a global function. One of these conditions is true:

- The main Blade layout does not have `@routes` before `@vite`.
- The component does not import `route` or `useRoute` from `ziggy-js`.
- The Ziggy package is not installed.
- The deployed assets are old.

17.2 Correction in the React component

Open the login component. The usual path is `resources/js/Pages/Auth/Login.jsx` or `Login.tsx`. Then use the hook:

```
import { useRoute } from 'ziggy-js';

export default function Login() {
    const route = useRoute();

    // Example:
    // form.post(route('login'));
}
```

As an alternative, import the helper directly in each module that uses it:

```
import { route } from 'ziggy-js';

// Example:
form.post(route('login'));
```

17.3 Correction with the global Blade helper

Use this method when the application uses `route()` as a global function. Add `@routes` before the Vite scripts in `resources/views/app.blade.php`:

```
<head>
    <!-- other tags -->
    @routes
    @viteReactRefresh
    @vite(['resources/js/app.jsx'])
    @inertiaHead
</head>
```

Use the correct entry file of the project, such as `app.js`, `app.jsx`, `app.ts`, or `app.tsx`.

> **CAUTION** — The directive `@routes` must be before the JavaScript bundle. The global helper is not available when the order is wrong.

17.4 Checks of Ziggy and of the route names

```
Set-Location C:\Sites\ats

composer show tightenco/ziggy
npm list ziggy-js
php artisan route:list --name=login
php artisan optimize:clear
```

Install the package when Composer reports that Ziggy is absent:

```
composer require tightenco/ziggy
```

Install or configure the JavaScript package when `ziggy-js` is absent.

17.5 New build of the production assets

```
Set-Location C:\Sites\ats
npm install
npm run build
php artisan optimize:clear
iisreset
```

Then do these checks:

1. Make sure that `public\build\manifest.json` has a new modification time.
2. Make sure that the JavaScript files with a hash in the name also have a new time.

3. Refresh the browser with Ctrl+F5, or use a private window.

17.6 Quick diagnosis in the browser

1. Show the page source. Find the word `Ziggy` or the text `route(`.
2. Make sure that the route configuration is before the Vite script when `@routes` is active.
3. Open the browser console. Run the command `typeof route`.
4. A global setup gives the result `"function"`. The result `"undefined"` shows that the helper is absent.
5. Open the login source component. Find each use of `route(`.
6. Make sure that each module imports the helper, uses `useRoute()`, or uses a correct global helper.

> **WARNING** — Do not edit a built file with a hash in its name, for example `login-r1iuorID.js`. Vite makes these files again and removes your changes.

17.7 Recommended sequence for the correction

1. Examine the login React component. Find each use of `route(...)`.
2. Add `import { useRoute } from 'ziggy-js'` in the React components. As an alternative, use `import { route } from 'ziggy-js'`.
3. Make sure that `tightenco/ziggy` is installed and that the login route has a name.
4. Put `@routes` before `@vite` in `resources/views/app.blade.php` for the global method.
5. Run `npm install` and `npm run build`, or copy a new `public/build` folder.
6. Run `php artisan optimize:clear`, then run `iisreset`, then refresh the browser.

18. Diagnostic Locations

Log or tool	Location or command
Laravel log	C:\Sites\ats\storage\logs\laravel.log
PHP error log	C:\Logs\php-errors.log
IIS access logs	C:\inetpub\logs\LogFiles
Windows events	Event Viewer > Windows Logs > Application and System
IIS Failed Request Tracing	Turn it on at the site level for difficult 4xx and 5xx errors
MariaDB error log	C:\Program Files\MariaDB 11.4\data*.err
MariaDB status	`Get-Service MariaDB`
Loaded php.ini	`php --ini`
PHP extensions	`php -m`
Laravel environment	`php artisan about`
Database connection	`php artisan db:show`
Route table	`php artisan route:list`

19. Security Checklist

☐ The IIS physical path ends with `public`. It is not the project root.
☐ `APP_ENV=production` and `APP_DEBUG=false`.
☐ The `.env` file is outside the public web root. It is not in source control.
☐ Each application has its own application pool and pool identity.
☐ The pool identity has Read and Execute on the code, and Modify only where necessary.
☐ Directory browsing is off.
☐ The files `test.php` and `phpinfo.php` are deleted.
☐ The HTTPS certificate and the binding are valid. HTTP goes to HTTPS only after the tests.
☐ Windows, IIS, PHP, Composer packages, and MariaDB have the current patches.
☐ The MariaDB account has privileges on the application database only.
☐ The MariaDB server uses `bind-address = 127.0.0.1` when it is on the application server.
☐ The MariaDB `root` account has a strong password and no remote access.
☐ The anonymous MariaDB accounts and the test database are removed.
☐ The production packages are installed with `--no-dev`.
☐ The development server is not in operation. The compiled assets are deployed.
☐ The database and the uploaded files have backups. The restore procedure is tested.
☐ The queue workers are supervised. They restart after each deployment.
☐ The scheduler operates each minute with an account with minimum privileges.
☐ The route `/up`, the logs, the disk capacity, the certificate date, and the backups are examined.
☐ A plan exists to replace Windows Server 2016 before 12 January 2027.

20. Final Validation Checklist

Phase	Complete when...
Server	IIS, CGI/FastCGI, URL Rewrite, PHP 8.3 x64 NTS, the VC++ runtime, Composer, and MariaDB are installed.
Database	The database, the account, and the privileges exist. `php artisan db:show` gives a result.
Application	The code, the packages, the built assets, the production `.env`, and `APP_KEY` are present.
Permissions	The pool identity reads the code and writes only to `storage`, `bootstrap\cache`, and the upload folders.
IIS	The pool uses No Managed Code. The site path ends with `public`. The bindings and the PHP handler operate.
Laravel	The migrations are complete. The storage link exists. The optimization is complete. Debug is off.
Operations	HTTPS, the scheduler, the queues, the backups, the logs, and the health check are tested.

☐ The root URL loads without a 401, 403, 404, or 500 error.
☐ Laravel controls `/login`. IIS does not treat it as a folder.
☐ The static CSS and JavaScript files give HTTP 200.
☐ The browser console does not show the error "route is not defined".
☐ The login POST request reaches Laravel.
☐ Correct credentials give a redirect to the dashboard.

☐ Laravel reads and writes the cache data and the session data.
☐ The MariaDB privileges stay limited to the application database.

21. References

No.	Source
1	Laravel 13 — Deployment
2	Laravel 13 — Installation and environment configuration
3	PHP Manual — Manual installation of pre-built Windows binaries
4	PHP Manual — Installation with IIS for Windows
5	Microsoft Learn — Using the URL Rewrite Module
6	Microsoft Learn — FastCGI configuration
7	Microsoft Lifecycle — Windows Server 2016
8	Microsoft Learn — Install-WindowsFeature
9	Microsoft Learn — CGI role service
10	Microsoft Learn — Troubleshoot HTTP 500.19
11	Microsoft Learn — PHP settings for IIS and FastCGI
12	Microsoft Learn — Application Pool Identities
13	Composer — Windows installation
14	MariaDB Documentation — CREATE USER, GRANT, and account management
15	MariaDB Documentation — mariadb-dump (mysqldump) and backup methods
16	MariaDB Documentation — Configuring MariaDB with option files
17	Laravel — Vite asset bundling
18	Tighten — Ziggy documentation for the JavaScript route helper
19	Inertia.js — Routing
20	ASD-STE100 — Simplified Technical English Specification

SCOPE NOTE — This manual gives a direct IIS and FastCGI deployment method with MariaDB. The exact settings for mail, cache, file storage, Active Directory, and the network depend on the application and on the institutional environment.